

# Networking Primer - Cheat Sheet

---

Backend services live and die on networking. Every database call, every API request, every Kafka publish is bytes over TCP. When latency spikes, when connections drop, when DNS suddenly returns the wrong answer, the difference between fixing it in 10 minutes and 4 hours is knowing what's underneath.

---

## Layered models

---

The OSI model has 7 layers. Nobody actually uses all 7 in conversation. The TCP/IP model (4 layers) is closer to reality.

| OSI layer      | TCP/IP layer              | Protocols                  |
|----------------|---------------------------|----------------------------|
| 7 Application  | Application               | HTTP, gRPC, DNS, SMTP, SSH |
| 6 Presentation | (folded into Application) | TLS, JSON, protobuf        |
| 5 Session      | (folded into Application) | TLS sessions, WebSocket    |
| 4 Transport    | Transport                 | TCP, UDP, QUIC             |
| 3 Network      | Internet                  | IP, ICMP, BGP              |
| 2 Data Link    | Link                      | Ethernet, ARP, Wi-Fi       |
| 1 Physical     | Link                      | Cables, radio              |

The two terms you'll hear in interviews are "L4" (transport, TCP/UDP) and "L7" (application, HTTP). Load balancers and proxies operate at one or the other.

---

## IP addressing

---

**IPv4:** 32-bit address, written as four octets (e.g., `192.168.1.42`). About 4.3 billion possible addresses. Ran out around 2011.

**IPv6:** 128-bit address, written as eight hex groups (e.g., `2001:db8::1`). 340 undecillion possible addresses. Adoption is slow but growing.

## CIDR notation

`192.168.1.0/24` means "this subnet uses the first 24 bits as the network part." Equivalent to subnet mask `255.255.255.0`.

| Prefix | Hosts | Common use              |
|--------|-------|-------------------------|
| /8     | 16M   | A whole class A network |
| /16    | 65K   | Typical company campus  |
| /24    | 254   | Common office subnet    |
| /29    | 6     | Point-to-point links    |
| /32    | 1     | Single host             |

## Private vs public ranges

These IP ranges don't route on the public internet:

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16
- 127.0.0.0/8 (loopback)

Most production VPCs use one of these. NAT translates them to public IPs at the gateway.

## TCP vs UDP

| Property    | TCP                           | UDP                                |
|-------------|-------------------------------|------------------------------------|
| Connection  | Connection-oriented           | Connectionless                     |
| Reliability | Ordered, retransmitted        | Best-effort                        |
| Speed       | Slower (overhead)             | Faster                             |
| Header      | 20+ bytes                     | 8 bytes                            |
| Use case    | HTTP, SSH, database protocols | DNS, video streaming, gaming, QUIC |

TCP is the default for everything backend. UDP shows up for DNS, streaming, and as the substrate under QUIC (HTTP/3).

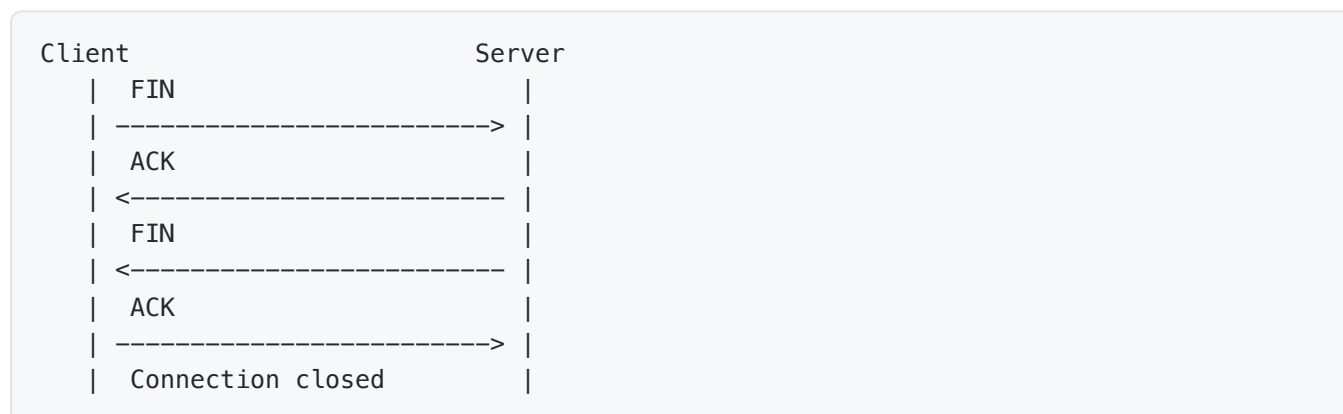
## The TCP handshake and close

### Three-way handshake (connection setup)



Three packets before any data flows. One round trip of latency before you send anything. TLS adds another. HTTP/3 over QUIC collapses these.

### Four-way close



After the active closer sends the final ACK, it stays in `TIME_WAIT` for  $2 * \text{MSL}$  (maximum segment lifetime, typically 60-120s). Reusing the same port too quickly produces `Address already in use` errors.

## Connection states (visible in `netstat` / `ss` )

| State                                             | What it means                        |
|---------------------------------------------------|--------------------------------------|
| <code>LISTEN</code>                               | Server waiting for connections       |
| <code>SYN_SENT</code>                             | Client sent SYN, waiting for SYN-ACK |
| <code>SYN_RECEIVED</code>                         | Server sent SYN-ACK, waiting for ACK |
| <code>ESTABLISHED</code>                          | Connection active, data flowing      |
| <code>FIN_WAIT_1</code> / <code>FIN_WAIT_2</code> | Active closer waiting for peer's FIN |
| <code>CLOSE_WAIT</code>                           | Peer closed, we haven't yet          |
| <code>TIME_WAIT</code>                            | Waiting to clean up after close      |
| <code>CLOSED</code>                               | No connection                        |

Lots of `CLOSE_WAIT` connections means the app isn't closing sockets. Connection pool leak.

Lots of `TIME_WAIT` means you're opening and closing too many short-lived connections. Use a pool or `SO_REUSEADDR` .

---

## DNS

DNS translates names to addresses. The protocol you take for granted until it fails.

## Record types

| Record | Maps                                             |
|--------|--------------------------------------------------|
| A      | Name -> IPv4 address                             |
| AAAA   | Name -> IPv6 address                             |
| CNAME  | Name -> another name (alias)                     |
| MX     | Name -> mail server                              |
| TXT    | Name -> arbitrary text (SPF, DKIM, verification) |
| NS     | Name -> authoritative name servers               |
| SOA    | Zone metadata (start of authority)               |
| SRV    | Name -> service location (host + port)           |
| PTR    | IP -> name (reverse lookup)                      |

## Resolution flow

```
client -> resolver (e.g., 8.8.8.8) -> root NS -> TLD NS -> authoritative NS
                                     <-           <-           <-
client <- resolver
```

The resolver does most of the work and caches answers. Cached responses respect the TTL.

## TTL surprises

DNS TTLs decide how stale cached answers can be. A 300s TTL means clients can take up to 5 minutes to see a DNS change. Many JVMs cache DNS forever ( `networkaddress.cache.ttl` ). Set it to something sensible (30s) for production.

---

## HTTP

The protocol the web is built on. Three versions matter today.

### HTTP/1.1

Plain text. Each connection handles one request at a time (or pipelined, but pipelining never worked well in practice). Persistent connections via `Connection: keep-alive` reuse the TCP socket. Adds the `Host` header so one IP can serve many domains.

## HTTP/2

Binary. Multiplexed: many requests share one TCP connection without head-of-line blocking at the HTTP layer. Header compression via HPACK. Server push (deprecated, never caught on).

The big win: a single connection per origin, all requests in parallel. Latency drops dramatically when a page has lots of small assets.

## HTTP/3

Built on QUIC, which runs over UDP. Solves TCP's head-of-line blocking at the transport layer. TCP retransmits block the whole stream; QUIC keeps streams independent. Built-in TLS 1.3. 0-RTT for resumed connections.

Faster on lossy networks (mobile). Slow to roll out widely because of UDP filtering in some networks.

## Methods

| Method  | Use                                 | Idempotent |
|---------|-------------------------------------|------------|
| GET     | Read                                | Yes        |
| HEAD    | Read headers only                   | Yes        |
| POST    | Create or non-idempotent action     | No         |
| PUT     | Replace                             | Yes        |
| PATCH   | Partial update                      | Sometimes  |
| DELETE  | Remove                              | Yes        |
| OPTIONS | What methods does this URL support? | Yes        |
| CONNECT | Tunnel (used by HTTPS proxies)      | No         |

## Status code families

| Range | Meaning       |
|-------|---------------|
| 1xx   | Informational |
| 2xx   | Success       |
| 3xx   | Redirect      |
| 4xx   | Client error  |
| 5xx   | Server error  |

(See `rest-api.pdf` for the full status code list with usage notes.)

---

## HTTPS and TLS

---

### TLS handshake (simplified, TLS 1.2)

```
Client -> Server: ClientHello (TLS version, ciphers, random)
Server -> Client: ServerHello + certificate + ServerHelloDone
Client: verify cert chain, generate session key
Client -> Server: encrypted pre-master secret
Both:   derive symmetric session key
Both:   encrypted application data
```

TLS 1.2 takes two round trips. TLS 1.3 cuts it to one, and adds 0-RTT for resumed sessions.

### Certificate validation

The client checks:

1. Domain name matches the cert's CN or SAN.
2. Cert is signed by a trusted CA (cert chain back to a root in the trust store).
3. Cert hasn't expired.
4. Cert hasn't been revoked (OCSP, CRL).

### SNI

Server Name Indication: a TLS extension that lets the client tell the server which hostname it wants during the handshake. Without SNI, one IP could only serve one TLS cert. With SNI, many domains share an IP.

---

## WebSockets

---

Persistent bidirectional connection. Starts as an HTTP request with `Upgrade: websocket`, then the TCP socket stays open for full-duplex messages.

Use for chat, live notifications, collaborative editing, dashboards that need server-pushed updates. Skip them for one-shot request/response.

---

## Load balancers (L4 vs L7)

| Layer            | What it sees                               | Examples                                 |
|------------------|--------------------------------------------|------------------------------------------|
| L4 (transport)   | TCP/UDP connections, no HTTP understanding | AWS NLB, HAProxy in TCP mode             |
| L7 (application) | HTTP method, headers, path, body           | nginx, Envoy, AWS ALB, GCP Load Balancer |

L7 is more flexible (route by path, header, cookie). L4 is faster and works for any TCP protocol.

(See `scaling-strategy.pdf` for the load balancing algorithms.)

## NAT, proxies, firewalls

### NAT (Network Address Translation)

Translates private IPs to a shared public IP. Your home router does this. Cloud VPC gateways do it in production. Without NAT, you'd need a public IP per device.

### Forward proxy

Sits between a client and the internet. Used for filtering, logging, caching outbound traffic. The proxy makes the outbound request on the client's behalf.

### Reverse proxy

Sits in front of servers. Receives incoming traffic and forwards it to the right backend. nginx, Envoy, and HAProxy are reverse proxies. Most load balancers are reverse proxies with extra features.

### Firewall / security group

Filters packets based on rules (source IP, dest port, protocol). Cloud security groups are stateful firewalls scoped per instance or per VPC.

Default deny plus explicit allow is the safe pattern.

## CORS (Cross-Origin Resource Sharing)

Browser security. By default, JavaScript on `https://a.com` can't read responses from `https://b.com`. The server at `b.com` opts in by sending the right `Access-Control-Allow-*` headers.

For “simple” requests (GET or POST with safe headers), the response just needs `Access-Control-Allow-Origin` . For others, the browser sends a preflight `OPTIONS` request first and checks `Access-Control-Allow-Methods` and `Access-Control-Allow-Headers` .

CORS lives in the browser. The server opts in via headers. A misconfigured CORS blocks legitimate clients but doesn't expose any data the server wouldn't otherwise serve.

---

## Common ports

| Port  | Service                           |
|-------|-----------------------------------|
| 22    | SSH                               |
| 25    | SMTP                              |
| 53    | DNS                               |
| 80    | HTTP                              |
| 110   | POP3                              |
| 143   | IMAP                              |
| 443   | HTTPS                             |
| 465   | SMTPS                             |
| 587   | SMTP (submission)                 |
| 993   | IMAPS                             |
| 995   | POP3S                             |
| 3306  | MySQL                             |
| 5432  | PostgreSQL                        |
| 6379  | Redis                             |
| 8080  | HTTP alt (Tomcat / Jetty default) |
| 9092  | Kafka                             |
| 27017 | MongoDB                           |

---

## Troubleshooting tools

---

The order I reach for them when something's broken.

### Reachability and routing

```
ping example.com           # ICMP, basic reachability
ping -c 4 8.8.8.8         # test by IP to rule out DNS

traceroute example.com     # path to destination
traceroute -T -p 443 host  # TCP traceroute (some firewalls block ICMP)
```

### DNS

```
dig example.com           # standard query
dig +trace example.com    # full resolution path
dig example.com MX        # query specific record type
dig @8.8.8.8 example.com  # ask a specific resolver

nslookup example.com      # old standby
```

### HTTP

```
curl -v https://example.com/api/users    # full request/response trace
curl -I https://example.com              # headers only (HEAD)
curl -X POST -d '{"x":1}' -H 'Content-Type: application/json' https://example.com/api
curl -w '%{time_total}\n' -o /dev/null -s https://example.com # measure latency
```

### TCP

```
ss -tnlp           # listening TCP ports + the process
ss -tn state established # current TCP connections
netstat -tan        # older, same idea

nc -zv host 443     # test if a port is open
telnet host 443     # older, same idea (Ctrl-J then quit)

lsof -i :8080        # what's bound to port 8080
```

### Packet capture

```
tcpdump -i any -nn 'port 5432'           # capture Postgres traffic
tcpdump -i any -nn -w out.pcap 'host 1.2.3.4' # write to file for Wireshark
```

Wireshark is the GUI for reading those pcaps.

---

## Backend networking gotchas

---

- **DNS caching in the JVM.** The default TTL is forever for some lookups. Set `networkaddress.cache.ttl` and `networkaddress.cache.negative.ttl` explicitly (30s is sensible).
  - **Connection pool sized too small.** Requests queue waiting for a connection. Latency jumps.
  - **Connection pool sized too large.** Hits database `max_connections`. Cascading failure.
  - **Keep-alive timeout mismatch.** Client keeps a connection open for 60s, load balancer closes after 30s. Client sees random connection resets.
  - **No connect timeout set.** A slow DNS or a SYN that gets no response holds your thread forever.
  - **TLS handshake adds latency.** Especially on cold connections. Reuse connections.
  - **NAT timeout for idle connections.** Cloud NAT gateways drop idle connections after a few minutes. Long-lived connections need TCP keepalive packets.
  - **TIME\_WAIT exhaustion.** A high-traffic client opening short-lived connections runs out of ephemeral ports. Use a connection pool.
  - **Half-open connections.** The peer crashed but TCP doesn't know. The sender notices only when it tries to write. Enable TCP keepalive or use application-level heartbeats.
  - **Local development with `localhost` vs `127.0.0.1`.** Java's IPv6 preference can make `localhost` resolve to `::1` while your app binds to `127.0.0.1`. Result: connection refused.
- 

## Best practices

---

- Always set connect and request timeouts on every external call.
  - Use connection pools. Don't open a fresh TCP connection per request.
  - Cache DNS, but not forever. 30s for positive results is a sensible default.
  - Prefer HTTP/2 or HTTP/3 for high-volume APIs to amortize handshake cost.
  - TLS everywhere, even inside the VPC, if you can afford it. Service mesh makes it easy.
  - Default-deny security groups. Open only the ports you need.
  - Monitor connection counts and pool saturation, not just request rate.
  - Trust `curl -v` and `dig` output when debugging. They tell you what the network actually saw.
  - Health checks should exercise the real path (DNS + TCP + TLS + HTTP), not just `127.0.0.1`.
  - Pin TLS versions. TLS 1.2 minimum. TLS 1.3 when your dependencies support it.
- 

## Common gotchas

---

- **Loopback isn't tested in production.** A health check that hits `localhost:8080` passes even when DNS or external networking is broken.

- **IPv4-only assumptions.** A library that doesn't support IPv6 fails when run in modern Kubernetes (which is IPv6-friendly).
  - **SNI required for shared IPs.** An old client without SNI gets the default cert, which doesn't match the hostname, and the handshake fails.
  - **Self-signed certs in non-prod.** Forgetting to trust them gives `SSLHandshakeException` that doesn't say what's wrong.
  - **`netstat -tan` shows all connections, including `TIME_WAIT`.** A pile of those isn't necessarily a problem unless you're running out of ports.
  - **Container DNS quirks.** Docker uses `/etc/resolv.conf` from the host. Kubernetes uses CoreDNS. Both add caching at the wrong layer sometimes.
  - **MTU mismatches.** Path MTU discovery is supposed to handle this. ICMP blocking breaks it, leading to "some requests work, some hang" symptoms.
  - **CORS preflight cached too long.** A bad CORS config cached by browsers for 24 hours wastes your day until you clear caches.
  - **Public IP changes during a session.** Mobile networks rotate IPs. Long-lived sticky sessions break.
-

## Quick reference

| Concept               | Key fact                                                             |
|-----------------------|----------------------------------------------------------------------|
| OSI model             | 7 layers (rarely all used in conversation)                           |
| TCP/IP model          | 4 layers, closer to reality                                          |
| L4 vs L7              | Transport (TCP/UDP) vs Application (HTTP)                            |
| IPv4 / IPv6           | 32-bit dotted vs 128-bit hex                                         |
| CIDR                  | <code>/24</code> means 24 prefix bits (256 addresses)                |
| Private IP ranges     | <code>10/8</code> , <code>172.16/12</code> , <code>192.168/16</code> |
| TCP handshake         | SYN, SYN-ACK, ACK (3 packets, 1 RTT)                                 |
| TIME_WAIT             | 2 * MSL after close (typically 60-120s)                              |
| DNS records           | A, AAAA, CNAME, MX, TXT, NS, SOA, SRV, PTR                           |
| HTTP versions         | 1.1 (text, persistent), 2 (binary, mux), 3 (over QUIC)               |
| TLS 1.2 vs 1.3        | 2 RTT vs 1 RTT, 1.3 adds 0-RTT                                       |
| SNI                   | TLS extension for hostname-based cert selection                      |
| <code>dig</code>      | Modern DNS query tool                                                |
| <code>curl -v</code>  | Full HTTP request/response trace                                     |
| <code>ss -tnlp</code> | Listening TCP ports + processes                                      |
| <code>tcpdump</code>  | Packet capture for offline analysis                                  |
| Common ports          | 80 HTTP, 443 HTTPS, 22 SSH, 5432 Postgres, 6379 Redis                |